>_ MCMP113 · Week 7

Loops: `while` and `do-while`

Pre-test and post-test loops, sentinels, input validation, menus and number algorithms

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 7 slides and add what does not fit on a slide: the exact behaviour of `while` and `do-while`, flowcharts, trace tables, all twelve example programs with real sample runs, the common mistakes, a glossary and graded exercises. Every listing is the tested program from the `code` folder (`07_01` ... `07_12`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. Read Sections 3–5 before the lab, then work through the examples and the exercises in Section 15. Next week (Week 8) is the midterm review: everything in these notes is exam material.

Contents

1 Learning outcomes **2**

2 When we cannot count: from `for` to `while` **3**

3 The `while` loop **3**

3.1 Tracing a `while` loop . . . 3

4 The `do-while` loop **4**

4.1 Flowcharts: where is the diamond? . . . 5

4.2 Example 1 – one countdown, three loops . . . 5

5 Choosing a loop; converting `for` to `while` **6**

5.1 Every `for` loop is a `while` loop in disguise . . . 6

5.2 Which loop when? . . . 7

6 Sentinel-controlled loops **7**

6.1 Example 2 – class average with the sentinel −1 . . . 8

7 Input validation loops **9**

7.1 Example 3 – ask again until the input is valid . . . 9

8 Menu-driven programs **10**

8.1 Example 4 – Okan Mini Bank . . . 11

9 Number algorithms with `while` **13**
9.1 Example 5 – Euclid's GCD (Week 1) and the LCM . . . 13
9.2 Peeling off digits: `% 10` and `/ 10` . . . 14
9.3 Example 6 – digit count, digit sum, reverse, palindrome . . . 14
9.4 Example 7 – decimal to binary . . . 15
9.5 Example 8 – the Collatz sequence . . . 16

10 Games, attempts and loop control **18**
10.1 `while (true)`, `break` and `continue` . . . 18
10.2 Example 9 – guess the number with `Random` . . . 18
10.3 Example 10 – the computer guesses by halving . . . 19
10.4 Example 11 – password with three attempts . . . 20
10.5 Example 12 – how many years until the money doubles? . . . 21

11 Infinite loops and loop invariants **22**
11.1 Infinite loops . . . 22
11.2 Loop invariants (informally) . . . 23

12 Common mistakes **23**

13 Summary **24**

14 Glossary (English – Turkish) **25**

15 Exercises **26**
15.1 Check yourself: predict the output . . . 26
15.2 In-class exercises (pairs) . . . 26
15.3 Homework . . . 26
15.4 Extra practice . . . 27

16 References **27**

>_ 1 Learning outcomes

After this week you should be able to:

1. write `while` loops (pre-test) and `do-while` loops (post-test) and explain why the first may run zero times and the second at least once;
2. draw the flowchart of both loops and trace them by hand in a trace table;
3. choose between `for`, `while` and `do-while` (counter-controlled vs. condition-controlled repetition) and convert a `for` loop into a `while` loop;
4. read data until a **sentinel** value, and validate input with a loop and `TryParse`;
5. build a menu-driven program with `do-while` and `switch`;

6. process the digits of an integer with `% 10` and `/ 10`, and implement Euclid's GCD, the LCM, decimal-to-binary conversion and the Collatz sequence;
7. use `while (true)` with `break`, and `continue`; recognise infinite loops and state a simple loop invariant.

>_ 2 When we cannot count: from `for` to `while`

In Week 6 every loop was **counter-controlled**: before the first pass we knew exactly how many passes there would be ("print 1 to n", "read 5 grades"). Many everyday problems are different:

- a teacher types exam scores until she types -1 — the program cannot know how many scores will come;
- a guessing game goes on until the guess is correct;
- a bank deposit grows until it has doubled — how many years is that?

In each case we know *when to stop*, not *how many times* to repeat. Such repetition is called **condition-controlled**. In Week 1 we already wrote it in pseudocode with **WHILE** (Algorithm 5, Euclid's GCD, and the guess-the-number game). This week we write it in C#.

>_ 3 The `while` loop

Definition – while statement

`while (condition) { body }` The *condition* (a `bool` expression) is tested **before** every pass. If it is `true`, the body runs once and the condition is tested again; if it is `false`, execution continues after the loop. Because the test comes first, the `while` loop is a **pre-test loop**: its body runs $0, 1, 2, \ldots$ times.

```
while (condition)
{
    // body: runs again and again
    // as long as condition is true
}
// next statement after the loop
```

Three things must be arranged by the programmer (in a `for` loop the header did it for us):

1. **initialise** the variables used in the condition *before* the loop;
2. write the **condition** so that it is `true` exactly while there is more work to do;
3. make **progress** inside the body: some statement must change a variable of the condition, otherwise the condition never becomes `false` and the loop never ends (an **infinite loop**, Section 11).

3.1 Tracing a `while` loop

How many years does it take for 100 TL to pass 1000 TL if the money doubles every year?

```
int money = 100;
int years = 0;
while (money < 1000)
{
    money *= 2;
    years++;
}
Console.WriteLine($"{years} years, {money} TL");    // 4 years, 1600 TL
```

A **trace table** for a `while` loop has *one row per test of the condition*:

Test	money	years	money < 1000	Action
1	100	0	true	run body
2	200	1	true	run body
3	400	2	true	run body
4	800	3	true	run body
5	1600	4	false	exit loop

Two facts are worth remembering for the exam:

- the condition is tested **one time more** than the body runs (5 tests, 4 passes) — the last test is the one that fails;
- after the loop the variable has *passed* the limit: `money` is 1600, not 1000.

In general, if the money starts at m_0 and doubles every year, the loop stops after the smallest k with $m_0 \cdot 2^k \geq 1000$; for $m_0 = 100$ this is $k = \lceil \log_2 10 \rceil = 4$.

>_ 4 The `do-while` loop

Definition – do-while statement

`do { body } while (condition);` The body runs **first**; then the condition is tested. If it is `true` the body runs again. Because the test comes after the body, the `do-while` loop is a **post-test loop**: its body runs $1, 2, 3, \ldots$ times — **never zero**.

```
do
{
    // body: runs FIRST,
    // then the condition is tested
}
while (condition);   // semicolon!
```

Two syntax details:

- The line `while (condition);` **ends with a semicolon**. Forgetting it gives the compiler error `CS1002: ; expected`. (In a normal `while` loop a semicolon after the condition is a *bug*, Section 11.)
- A variable declared *inside* the braces of `do { … }` does not exist any more in the condition `while (...)`. Declare the variables of the condition *before* the loop (see Example 3).

4.1 Flowcharts: where is the diamond?

Figure 1 shows both loops with the *same* boxes and the same start value $i = 0$. The only difference is the position of the decision diamond. In the `while` loop the diamond comes first, so with $i = 0$ the test $i > 0$ fails immediately and nothing is printed. In the `do-while` loop the body is executed before the first test, so 0 is printed once.

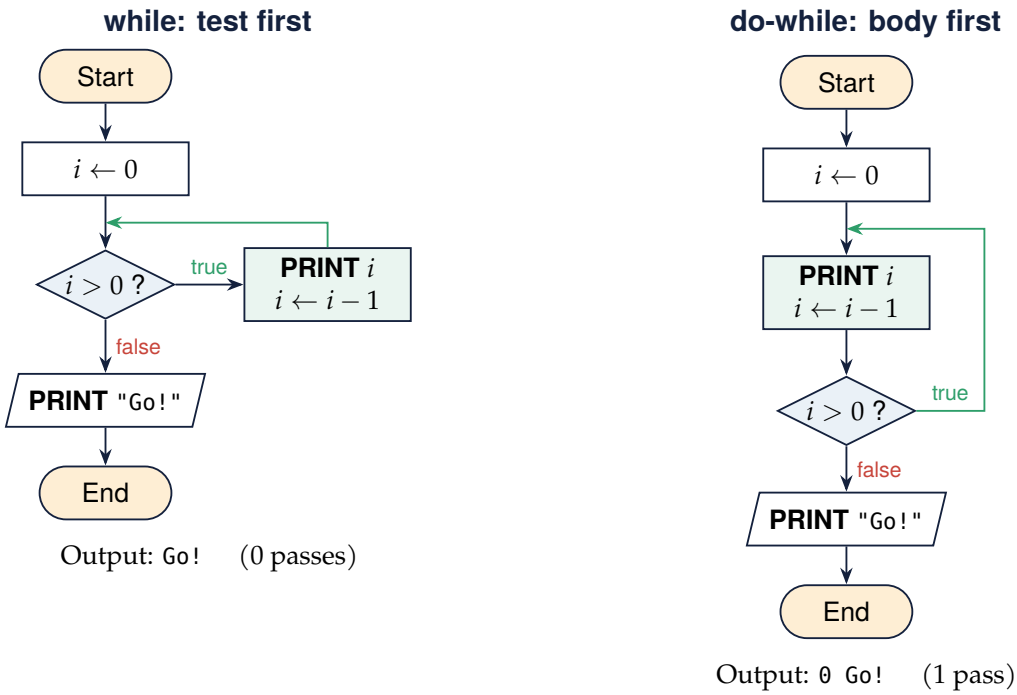


Figure 1: The same loop as pre-test (`while`) and post-test (`do-while`) with start value $i = 0$. The green back edge is the repetition.

Pseudocode – pre-test

```
i ← 0
WHILE i > 0
    PRINT i
    i ← i − 1
END WHILE
PRINT "Go!"
```

Pseudocode – post-test

```
i ← 0
REPEAT
   PRINT i
   i ← i − 1
WHILE i > 0
PRINT "Go!"
```

In pseudocode we write the post-test loop as **REPEAT** … **WHILE** *condition*, so that the test is visibly at the bottom.

4.2 Example 1 – one countdown, three loops

The first program counts down from n with all three loops. Run it with $n = 3$ and with $n = 0$.

Listing 1: 07_01_CountdownThreeWays/Program.cs

```
1  // MCMP113 · Week 7 · Example 1 — Countdown three ways
2  // The same countdown with while, do-while and for; try n = 0.
3
4  Console.Write("Start the countdown from: ");
5  int n = int.Parse(Console.ReadLine());
6
```

```
// 1) while: condition tested BEFORE each pass (pre-test)
Console.Write("while    : ");
int i = n;
while (i > 0)
{
    Console.Write($"{i} ");
    i--;
}
Console.WriteLine("Go!");

// 2) do-while: condition tested AFTER each pass (post-test)
Console.Write("do-while : ");
int j = n;
do
{
    Console.Write($"{j} ");
    j--;
}
while (j > 0);
Console.WriteLine("Go!");

// 3) for: the same counter-controlled loop, one header line
Console.Write("for      : ");
for (int k = n; k > 0; k--)
{
    Console.Write($"{k} ");
}
Console.WriteLine("Go!");
```

Sample run 1

```
Start the countdown from: 3
while    : 3 2 1 Go!
do-while : 3 2 1 Go!
for      : 3 2 1 Go!
```

Sample run 2

```
Start the countdown from: 0
while    : Go!
do-while : 0 Go!
for      : Go!
```

For $n \geq 1$ the three loops agree. For $n = 0$ only the `do-while` loop differs: it prints `0` before it tests `j > 0`. Which version is "right" depends on the problem — for a countdown, printing nothing from 0 is correct, so `while` or `for` fits better. `do-while` is neither better nor worse; it *means* something different.

>_ 5 Choosing a loop; converting `for` to `while`

5.1 Every `for` loop is a `while` loop in disguise

The three parts of a `for` header move to three different places in a `while` loop:

for

```
int total = 0;
for (int k = 1; k <= 5; k++)
{
    total += k;
}
```

while

```
int total = 0;
int k = 1;          // 1 init
while (k <= 5)      // 2 condition
{
    total += k;
    k++;            // 3 update
}
```

Part	for	while
1 initialisation	inside the header	a statement before the loop
2 condition	in the header	in `while ( … )`
3 update	in the header	usually the last statement of the body
`k` after the loop	does not exist (scope)	still exists, value 6

Both versions give `total = 15`. One subtle difference: a `continue` (Section 10) in a `for` loop still executes the update `k++`; in the `while` version a `continue` placed *before* `k++` skips the update and the loop never ends.

5.2 Which loop when?

All three loops can express every repetition; we choose the one that makes our *intent* obvious to the reader.

Situation	Example	Best choice
number of passes known before the loop	print $1..N$, read exactly N scores	`for`
repeat until a special stop value	read scores until -1 (sentinel)	`while`
zero passes must be possible	process digits while $n > 0$	`while`
body must run at least once	menu, "ask again" input validation	`do-while`
stop decision in the middle of the body	guess game: read, check, `break`	`while (true)` + `break`

>_ 6 Sentinel-controlled loops

Definition – sentinel

A **sentinel** is a special input value that cannot be real data and only means "stop" (e.g. -1 for exam scores, 0 for a list of non-zero numbers). A loop that reads until the sentinel is a **sentinel-controlled loop**.

The pattern needs **two** READ statements: the **priming read** before the loop, and a second READ as the *last* statement of the body. Thus every value is tested against the sentinel *before* it is used, and the sentinel itself is never added to the sum.

Algorithm 1 – Average until -1

```
START
count ← 0; sum ← 0
READ score                          (priming)
WHILE score ≠ −1
    count ← count + 1
    sum ← sum + score
    READ score                      (next)
END WHILE
IF count > 0 THEN
    PRINT sum / count
ELSE
    PRINT "No scores"
END IF
END
```

READ #	score	score $\neq -1$	count	sum
1 (priming)	72	true	1	72
2	85	true	2	157
3	90	true	3	247
4	64	true	4	311
5	-1	false	4	311

$$\text{average} = 311/4 = 77.75$$

6.1 Example 2 – class average with the sentinel -1

Listing 2: 07_02_SentinelAverage/Program.cs

```
// MCMP113 · Week 7 · Example 2 — Sentinel-controlled average
// Reads scores until the sentinel -1; prints count, average, highest.

const int Sentinel = -1;
int count = 0;
int sum = 0;
int highest = 0;

Console.WriteLine($"Enter scores (0-100), {Sentinel} to finish.");
Console.Write("Score: ");
int score = int.Parse(Console.ReadLine());  // priming read

while (score != Sentinel)
{
    count++;
    sum += score;
    if (count == 1 || score > highest)
    {
        highest = score;        // first score or a new record
    }
    Console.Write("Score: ");
    score = int.Parse(Console.ReadLine());  // the NEXT value
}

if (count == 0)
{
    Console.WriteLine("No scores entered.");
}
else
{
    double average = (double)sum / count;
    Console.WriteLine($"Count: {count}  Sum: {sum}");
    Console.WriteLine($"Average: {average:F2}  Highest: {highest}");
}
```

Sample run 1

```
Enter scores (0-100), -1 to finish.
Score: 72
Score: 85
Score: 90
Score: 64
Score: -1
Count: 4  Sum: 311
Average: 77.75  Highest: 90
```

Sample run 2

```
Enter scores (0-100), -1 to finish.
Score: -1
No scores entered.
```

Notes on the program:

- The sentinel is a named constant (`const int Sentinel = -1;`, Week 3): to change it we edit one line.
- `if (count == 1 || score > highest)` makes the *first* score the initial record. Initialising `highest` with 0 alone would also work for scores 0..100, but not for data that can be negative.
- The `if (count == 0)` test after the loop matters: with no scores, `(double)sum / count` would be $0.0/0 =$ `NaN` ("not a number", Week 4) and the program would print `Average: NaN`.

>_ 7 Input validation loops

Users make typing mistakes. A **validation loop** asks again until the input is acceptable. Acceptable means two things: the text is a number (`TryParse` returns `true`, Week 5) **and** the number is in the allowed range. We repeat while the input is *not* valid; by De Morgan's law (Week 4)

$$\neg(ok \wedge 0 \le s \wedge s \le 100) \;=\; \neg ok \vee s < 0 \vee s > 100,$$

which in C# is `!ok || score < 0 || score > 100`.

7.1 Example 3 – ask again until the input is valid

Part (a) uses a `do-while` loop: we must ask at least once. Part (b) does the same with a `while` loop and a priming read.

Listing 3: 07_03_ValidatedInput/Program.cs

```
// MCMP113 · Week 7 · Example 3 — Input validation loops
// Keeps asking until the input is a number AND inside the allowed range.

// (a) do-while: ask first, check afterwards
int score;
bool ok;
do
{
    Console.Write("Exam score (0-100): ");
    ok = int.TryParse(Console.ReadLine(), out score);
    if (!ok || score < 0 || score > 100)
    {
        Console.WriteLine("  Invalid: whole number 0-100 only.");
    }
}
while (!ok || score < 0 || score > 100);

// (b) while with a priming read: read once, repeat while invalid
Console.Write("Height in metres (0.5-2.5): ");
```

```
20 bool valid = double.TryParse(Console.ReadLine(), out double height);
21 while (!valid || height < 0.5 || height > 2.5)
22 {
23     Console.WriteLine("  Invalid: example 1.75");
24     Console.Write("Height in metres (0.5-2.5): ");
25     valid = double.TryParse(Console.ReadLine(), out height);
26 }
27
28 Console.WriteLine($"Accepted: {score} points, {height:F2} m");
```

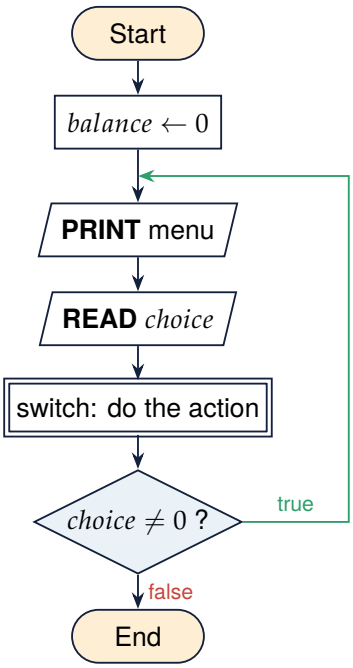
Sample run

```
Exam score (0-100): abc
  Invalid: whole number 0-100 only.
Exam score (0-100): 120
  Invalid: whole number 0-100 only.
Exam score (0-100): 85
Height in metres (0.5-2.5): 1,75
  Invalid: example 1.75
Height in metres (0.5-2.5): 0
  Invalid: example 1.75
Height in metres (0.5-2.5): 1.75
Accepted: 85 points, 1.75 m
```

- `score` and `ok` are declared *before* the `do` block, because the condition `while (...)` must see them.
- Why test `!ok` at all? When `TryParse` fails it sets the `out` variable to 0, and 0 is inside 0..100. Without `!ok` the text `abc` would be accepted as the score 0.
- The input `1,75`: our project template uses the invariant culture (Week 3), where the comma is a *thousands separator*. `TryParse` therefore succeeds with the value 175 — and the *range* check rejects it. This is a good reason to always check the range, too.

>_ 8 Menu-driven programs

A menu program shows a list of options, reads the user's choice, performs the action, and repeats until the user chooses "Exit". The menu must appear at least once, so the natural structure is a **`do-while` loop around a `switch`** (Week 5).



do-while: the menu is shown at least once.

switch: one `case` per option; `default` catches everything else.

Exit: the loop condition is `choice != 0`.

8.1 Example 4 – Okan Mini Bank

Listing 4: 07_04_BankMenu/Program.cs

```
// MCMP113 · Week 7 · Example 4 – Menu-driven bank account
// A do-while menu loop with a switch: the menu is shown at least once.

decimal balance = 0m;
int choice;
Console.WriteLine("=== OKAN MINI BANK ===");

do
{
    Console.WriteLine("1) Balance 2) Deposit 3) Withdraw 0) Exit");
    Console.Write("Choice: ");
    if (!int.TryParse(Console.ReadLine(), out choice))
    {
        choice = -1;        // not a number -> default case
    }

    switch (choice)
    {
        case 1:
            Console.WriteLine($"Balance: {balance:F2} TL");
            break;
        case 2:
            Console.Write("Amount to deposit: ");
            decimal deposit = decimal.Parse(Console.ReadLine());
            if (deposit > 0)
            {
                balance += deposit;
                Console.WriteLine($"New balance: {balance:F2} TL");
            }
            else
            {
                Console.WriteLine("The amount must be positive.");
            }
            break;
        case 3:
            Console.Write("Amount to withdraw: ");
```

```
            decimal amount = decimal.Parse(Console.ReadLine());
            if (amount <= 0)
            {
                Console.WriteLine("The amount must be positive.");
            }
            else if (amount > balance)
            {
                Console.WriteLine("Insufficient funds.");
            }
            else
            {
                balance -= amount;
                Console.WriteLine($"New balance: {balance:F2} TL");
            }
            break;
        case 0:
            Console.WriteLine("Goodbye!");
            break;
        default:
            Console.WriteLine("Unknown option, try again.");
            break;
    }
}
while (choice != 0);
```

Sample run

```
=== OKAN MINI BANK ===
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 2
Amount to deposit: 500
New balance: 500.00 TL
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 3
Amount to withdraw: 750
Insufficient funds.
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 3
Amount to withdraw: 200
New balance: 300.00 TL
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 1
Balance: 300.00 TL
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: x
Unknown option, try again.
1) Balance 2) Deposit 3) Withdraw 0) Exit
Choice: 0
Goodbye!
```

Pitfall – TryParse failure means 0 = Exit

When `int.TryParse` fails, `choice` becomes 0. Since 0 means "Exit", a typing mistake such as `x` would silently end the program. Example 4 therefore sets `choice = -1;` when the parse fails, so that the `default` case reports the error and the loop continues.

Money is stored in a `decimal` (Week 3). The amounts are read with `decimal.Parse`; validating them with a loop as in Example 3 is left as an exercise (H1 uses the same idea).

>_ 9 Number algorithms with `while`

9.1 Example 5 – Euclid's GCD (Week 1) and the LCM

Week 1's Algorithm 5 computed the greatest common divisor by repeating $(x, y) \leftarrow (y,\ x \bmod y)$ while $y \neq 0$. The number of passes depends on the data, so this is a `while` loop.

Algorithm 2 – Euclid

```
START
READ a, b
x ← a;  y ← b
WHILE y ≠ 0
    r ← x mod y
    x ← y
    y ← r
END WHILE
PRINT x
END
```

The C# program keeps copies `x` and `y` of the input, because `a` and `b` are still needed for the LCM. Inside the loop it prints one row of the trace table per pass, so the console output *is* the Week 1 trace table. The loop ends when $y = 0$; at that moment x is the GCD.
Invariant (Section 11): $\gcd(x, y) = \gcd(a, b)$ before every test, because $\gcd(x, y) = \gcd(y, x \bmod y)$.

Listing 5: 07_05_GcdLcm/Program.cs

```
// MCMP113 · Week 7 · Example 5 — Euclid's GCD and the LCM
// Week 1's algorithm in C#: (x, y) <- (y, x mod y) while y is not 0.

Console.Write("First positive integer a : ");
int a = int.Parse(Console.ReadLine());
Console.Write("Second positive integer b: ");
int b = int.Parse(Console.ReadLine());

if (a <= 0 || b <= 0)
{
    Console.WriteLine("Both numbers must be positive.");
}
else
{
    int x = a;          // copies: a and b are needed for the LCM
    int y = b;
    Console.WriteLine($"{"x",6} {"y",6} {"x % y",7}");
    while (y != 0)
    {
        int r = x % y;
        Console.WriteLine($"{x,6} {y,6} {r,7}");
        x = y;
        y = r;
    }
    int gcd = x;
    long lcm = (long)a / gcd * b;   // divide first, then multiply
    Console.WriteLine($"GCD({a}, {b}) = {gcd}");
    Console.WriteLine($"LCM({a}, {b}) = {lcm}");
}
```

```
Run: 48, 18

First positive integer a : 48
Second positive integer b: 18
     x      y   x % y
    48     18      12
    18     12       6
    12      6       0
GCD(48, 18) = 6
LCM(48, 18) = 144
```

```
Run: 18, 48

First positive integer a : 18
Second positive integer b: 48
     x      y   x % y
    18     48      18
    48     18      12
    18     12       6
    12      6       0
GCD(18, 48) = 6
LCM(18, 48) = 144
```

```
Run: 1071, 462

First positive integer a : 1071
Second positive integer b: 462
     x      y   x % y
  1071    462     147
   462    147      21
   147     21       0
GCD(1071, 462) = 21
LCM(1071, 462) = 23562
```

With $a < b$ the first pass simply swaps the numbers ($18 \bmod 48 = 18$). Input 0 or a negative number is rejected by the `if` before the loop (`Both numbers must be positive.`).

The LCM uses the identity $\gcd(a,b) \cdot \operatorname{lcm}(a,b) = a \cdot b$, so $\operatorname{lcm}(a,b) = \dfrac{a}{\gcd(a,b)} \cdot b$. We divide first ($a$/ gcd is exact) and compute in `long`, so the product cannot overflow an `int`. Check: $48/6 \cdot 18 = 144$ and $1071/21 \cdot 462 = 51 \cdot 462 = 23562$.

9.2 Peeling off digits: `% 10` and `/ 10`

For a non-negative integer n, `n % 10` is its last decimal digit and `n / 10` (integer division, Week 4) removes that digit: $4071 \bmod 10 = 1$, $4071 \operatorname{div} 10 = 407$. Repeating both until nothing is left visits every digit from right to left. The number of passes equals the number of digits — not known in advance, hence a condition-controlled loop. The statement `reversed = reversed * 10 + digit ;` shifts the digits collected so far one place to the left and appends the new one on the right: $1 \to 17 \to 170 \to 1704$.

rest	digit	digitCount	digitSum	reversed
4071	1	1	1	1
407	7	2	8	17
40	0	3	8	170
4	4	4	12	1704
0	**stop: `rest > 0` is false**			

The row shows `rest` at the start of a pass and the other values at the end of that pass.

9.3 Example 6 – digit count, digit sum, reverse, palindrome

Listing 6: 07_06_DigitTools/Program.cs

```
1  // MCMP113 · Week 7 · Example 6 — Digit tools
2  // % 10 gives the last digit, / 10 drops it: count, sum, reverse, palindrome.
3
4  int n;
5  bool ok;
6  do
7  {
8      Console.Write("Enter a non-negative integer: ");
9      ok = int.TryParse(Console.ReadLine(), out n);
10 }
11 while (!ok || n < 0);
12
13 int rest = n;           // working copy; n is kept for the test
14 int digitCount = 0;
15 int digitSum = 0;
```

```
long reversed = 0;      // long: the reverse may not fit in an int

// do-while, so that n = 0 is counted as a 1-digit number
do
{
    int digit = rest % 10;              // last digit
    digitCount++;
    digitSum += digit;
    reversed = reversed * 10 + digit;   // append on the right
    rest /= 10;                         // drop the last digit
}
while (rest > 0);

Console.WriteLine($"Digits : {digitCount}");
Console.WriteLine($"Sum    : {digitSum}");
Console.WriteLine($"Reverse: {reversed}");
if (reversed == n)
{
    Console.WriteLine($"{n} is a palindrome.");
}
else
{
    Console.WriteLine($"{n} is not a palindrome.");
}
```

Sample run 1

```
Enter a non-negative integer: 4071
Digits : 4
Sum    : 12
Reverse: 1704
4071 is not a palindrome.
```

Sample run 2

```
Enter a non-negative integer: -5
Enter a non-negative integer: 12321
Digits : 5
Sum    : 9
Reverse: 12321
12321 is a palindrome.
```

Sample run 3

```
Enter a non-negative integer: 0
Digits : 1
Sum    : 0
Reverse: 0
0 is a palindrome.
```

Sample run 4

```
Enter a non-negative integer: 2147483647
Digits : 10
Sum    : 46
Reverse: 7463847412
2147483647 is not a palindrome.
```

- **Why `do-while`?** The number 0 has one digit. A `while (rest > 0)` loop would run zero times and report `Digits : 0`.
- **Why `long reversed`?** The reverse of 2147483647 (`int.MaxValue`) is 7463847412, which is larger than `int.MaxValue` (run 4).
- **Why the copy `rest`?** The loop destroys its variable (it ends at 0); we still need the original n for the palindrome test.
- Leading zeros vanish: 1200 reverses to 21 (try it).

9.4 Example 7 – decimal to binary

The same idea in base 2: `rest % 2` is the last bit, `rest / 2` removes it. The remainders appear from the least significant bit to the most significant, so each new bit is written to the *left* of the string: `binary = bit + binary;`.

Listing 7: 07_07_DecimalToBinary/Program.cs

```
// MCMP113 · Week 7 · Example 7 — Decimal to binary
// Repeated division by 2; the remainders, read backwards, are the bits.

int n;
bool ok;
do
{
    Console.Write("Non-negative integer: ");
    ok = int.TryParse(Console.ReadLine(), out n);
}
while (!ok || n < 0);

int rest = n;
string binary = "";
do
{
    int bit = rest % 2;
    Console.WriteLine($"{rest} / 2 = {rest / 2}, remainder {bit}");
    binary = bit + binary;      // new bit goes to the LEFT
    rest /= 2;
}
while (rest > 0);

Console.WriteLine($"{n} in binary = {binary}");
Console.WriteLine($"Built-in check: {Convert.ToString(n, 2)}");
```

Sample run 1

```
Non-negative integer: 13
13 / 2 = 6, remainder 1
6 / 2 = 3, remainder 0
3 / 2 = 1, remainder 1
1 / 2 = 0, remainder 1
13 in binary = 1101
Built-in check: 1101
```

Sample run 2

```
Non-negative integer: 2026
2026 / 2 = 1013, remainder 0
1013 / 2 = 506, remainder 1
506 / 2 = 253, remainder 0
253 / 2 = 126, remainder 1
126 / 2 = 63, remainder 0
63 / 2 = 31, remainder 1
31 / 2 = 15, remainder 1
15 / 2 = 7, remainder 1
7 / 2 = 3, remainder 1
3 / 2 = 1, remainder 1
1 / 2 = 0, remainder 1
2026 in binary = 11111101010
Built-in check: 11111101010
```

Check: $1101_2 = 8+4+0+1 = 13$ and $11111101010_2 = 1024+512+256+128+64+32+8+2 = 2026$. Again `do-while` makes the input 0 print `0` instead of an empty string. `Convert.ToString(n, 2)` is the built-in conversion; we use it only to check our loop.

9.5 Example 8 – the Collatz sequence

Rule: if n is even, $n \leftarrow n/2$; if n is odd, $n \leftarrow 3n+1$; stop when $n = 1$. The **Collatz conjecture** (Lothar Collatz, 1937) states that every start value ≥ 1 eventually reaches 1. It has been checked by computer for enormous ranges of start values but has *never been proved*. So here is a loop whose number of passes nobody can predict — and nobody can even prove that it always terminates.

Listing 8: 07_08_Collatz/Program.cs

```
// MCMP113 · Week 7 · Example 8 — Collatz sequence
// even -> n / 2, odd -> 3n + 1, repeat until 1 (steps unknown in advance).

int start;
bool ok;
do
{
    Console.Write("Starting value (>= 1): ");
    ok = int.TryParse(Console.ReadLine(), out start);
}
while (!ok || start < 1);

long n = start;     // long: the values can grow a lot
int steps = 0;
long peak = n;
Console.Write(n);
while (n != 1)
{
    if (n % 2 == 0)
    {
        n = n / 2;
    }
    else
    {
        n = 3 * n + 1;
    }
    steps++;
    if (n > peak)
    {
        peak = n;
    }
    // 10 numbers per line keeps long sequences readable
    Console.Write(steps % 10 == 0 ? $"\n{n}" : $" {n}");
}
Console.WriteLine();
Console.WriteLine($"{steps} steps, highest value {peak}");
```

Sample run 1

```
Starting value (>= 1): 6
6 3 10 5 16 8 4 2 1
8 steps, highest value 16
```

Sample run 2

```
Starting value (>= 1): 0
Starting value (>= 1): 27
27 82 41 124 62 31 94 47 142 71
214 107 322 161 484 242 121 364 182 91
274 137 412 206 103 310 155 466 233 700
350 175 526 263 790 395 1186 593 1780 890
445 1336 668 334 167 502 251 754 377 1132
566 283 850 425 1276 638 319 958 479 1438
719 2158 1079 3238 1619 4858 2429 7288 3644 1822
911 2734 1367 4102 2051 6154 3077 9232 4616 2308
1154 577 1732 866 433 1300 650 325 976 488
244 122 61 184 92 46 23 70 35 106
53 160 80 40 20 10 5 16 8 4
2 1
111 steps, highest value 9232
```

The start value 27 needs 111 steps and climbs to 9232 before it falls to 1. We use `long` because intermediate values can be far larger than the start value. The conditional operator `?:` (Week 5) inserts a line break after every tenth number.

>_ 10 Games, attempts and loop control

10.1 `while (true)`, `break` and `continue`

- `break;` leaves the innermost loop *immediately* (we met it in `switch` and in Week 6).
- `continue;` skips the rest of the body and goes back to the test of the condition.
- `while (true)` has a condition that is never `false`; the only way out is a `break`. Use it when the stop decision is in the *middle* of the body (read — check — maybe stop — continue working). Every `while (true)` loop must contain a reachable `break`.

10.2 Example 9 – guess the number with `Random`

`Random.Shared.Next(1, 101)` returns a random integer from 1 to 100: the lower bound is *included*, the upper bound *excluded*. (A classic bug is `Next(1, 100)`, which can never produce 100.)

Listing 9: 07_09_GuessNumber/Program.cs

```
// MCMP113 · Week 7 · Example 9 — Guess the number
// Random secret, attempts counter, while (true) with break and continue.

int secret = Random.Shared.Next(1, 101);   // 1..100, 101 excluded
// Testing tip: a fixed seed gives the same secret in every run:
// int secret = new Random(42).Next(1, 101);
int attempts = 0;

Console.WriteLine("I am thinking of a number between 1 and 100.");
while (true)
{
    Console.Write("Your guess: ");
    if (!int.TryParse(Console.ReadLine(), out int guess))
    {
        Console.WriteLine("Please type a whole number.");
        continue;           // back to the top; attempt not counted
    }
    attempts++;
    if (guess < secret)
    {
        Console.WriteLine("Higher!");
    }
    else if (guess > secret)
    {
        Console.WriteLine("Lower!");
    }
    else
    {
        Console.WriteLine($"Correct! {attempts} attempts.");
        break;              // leave the loop immediately
    }
}
```

Testing a random program: fix the seed

Every run of the game has a different secret, so a run cannot be repeated. For testing we temporarily replace line 4 by the commented line 6, `int secret = new Random(42).Next(1, 101);`. A `Random` object created with the same **seed** produces the same sequence of numbers, so the secret is the same in every test run (with .NET 10 it is 67). The sample run below was produced exactly this way. Remove the seed again before "shipping" the game. (The concrete numbers produced by a seed are an implementation detail of .NET; real software must not rely on them.)

Test run with seed 42 (secret 67)

```
I am thinking of a number between 1 and 100.
Your guess: 50
Higher!
Your guess: 75
Lower!
Your guess: abc
Please type a whole number.
Your guess: 62
Higher!
Your guess: 68
Lower!
Your guess: 65
Higher!
Your guess: 66
Higher!
Your guess: 67
Correct! 7 attempts.
```

The input `abc` is reported, but because of `continue` it is not counted as an attempt. The player used the halving strategy of Week 1: always guess the middle of the remaining range.

10.3 Example 10 – the computer guesses by halving

Here the user chooses the secret and the computer plays the halving strategy — **binary search**. Because no randomness is involved, every run is reproducible.

Listing 10: 07_10_ComputerGuesses/Program.cs

```
1  // MCMP113 · Week 7 · Example 10 — The computer guesses by halving
2  // Week 1's binary-search strategy: never more than 7 guesses for 1..100.
3
4  int secret;
5  bool ok;
6  do
7  {
8      Console.Write("Secret number for the computer (1-100): ");
9      ok = int.TryParse(Console.ReadLine(), out secret);
10 }
11 while (!ok || secret < 1 || secret > 100);
12
13 int low = 1;
14 int high = 100;
15 int guesses = 0;
16 int guess;
17 do
18 {
19     guess = (low + high) / 2;       // middle of the range
```

```
    guesses++;
    Console.Write($"#{guesses}: {low}..{high} -> {guess}  ");
    if (guess < secret)
    {
        Console.WriteLine("higher");
        low = guess + 1;
    }
    else if (guess > secret)
    {
        Console.WriteLine("lower");
        high = guess - 1;
    }
    else
    {
        Console.WriteLine("correct!");
    }
}
while (guess != secret);

Console.WriteLine($"Found {secret} in {guesses} guesses.");
```

Sample run 1

```
Secret number for the computer (1-100): 67
#1: 1..100 -> 50  higher
#2: 51..100 -> 75  lower
#3: 51..74 -> 62  higher
#4: 63..74 -> 68  lower
#5: 63..67 -> 65  higher
#6: 66..67 -> 66  higher
#7: 67..67 -> 67  correct!
Found 67 in 7 guesses.
```

Sample run 2

```
Secret number for the computer (1-100): 150
Secret number for the computer (1-100): 100
#1: 1..100 -> 50  higher
#2: 51..100 -> 75  higher
#3: 76..100 -> 88  higher
#4: 89..100 -> 94  higher
#5: 95..100 -> 97  higher
#6: 98..100 -> 99  higher
#7: 100..100 -> 100  correct!
Found 100 in 7 guesses.
```

Running the program for every secret 1..100 gives the following distribution of the number of guesses:

guesses	1	2	3	4	5	6	7
secrets	1	2	4	8	16	32	37

Never more than 7 guesses, 5.80 on average $(580/100)$. After k guesses at most $2^k - 1$ secrets can have been found, so the worst case for 100 numbers is the smallest k with $2^k - 1 \geq 100$, i.e. $k = \lceil \log_2 101 \rceil = 7$. In Week 10 we apply binary search to sorted arrays.

10.4 Example 11 – password with three attempts

This loop has **two** ways to end: the password is correct (the flag `granted` becomes `true`) or no attempts are left (the counter reaches `MaxAttempts`). Hence the condition `attempts < MaxAttempts && !granted`. After the loop an `if` finds out *which* of the two ended it.

Listing 11: 07_11_PasswordAttempts/Program.cs

```
// MCMP113 · Week 7 · Example 11 — Password with 3 attempts
// Two exits from one loop: success (flag) or no attempts left (counter).

const string Password = "okan2026";
const int MaxAttempts = 3;
```

```
int attempts = 0;
bool granted = false;

while (attempts < MaxAttempts && !granted)
{
    Console.Write("Password: ");
    string entry = Console.ReadLine();
    attempts++;
    if (entry == Password)
    {
        granted = true;
    }
    else if (attempts < MaxAttempts)
    {
        int left = MaxAttempts - attempts;
        Console.WriteLine($"Wrong! {left} attempt(s) left.");
    }
}

if (granted)
{
    Console.WriteLine($"Access granted (attempt {attempts}).");
}
else
{
    Console.WriteLine("Wrong password. Account locked.");
}
```

Sample run 1

```
Password: okan2025
Wrong! 2 attempt(s) left.
Password: okan2026
Access granted (attempt 2).
```

Sample run 2

```
Password: 1234
Wrong! 2 attempt(s) left.
Password: password
Wrong! 1 attempt(s) left.
Password: Okan2026
Wrong password. Account locked.
```

String comparison with == is case-sensitive: `Okan2026` is wrong. (Real systems never store passwords in the source code; this program is only a loop exercise.)

10.5 Example 12 – how many years until the money doubles?

With an annual interest rate of r % the balance after one year is $B \cdot (1 + r/100)$. We repeat this while the balance is below twice the deposit; the number of years is not known in advance.

Listing 12: 07_12_DoublingTime/Program.cs

```
// MCMP113 · Week 7 · Example 12 – How many years until the money doubles?
// The number of passes is not known in advance -> a while loop.

Console.Write("Deposit (TL): ");
decimal principal = decimal.Parse(Console.ReadLine());
decimal rate;
do
{
    Console.Write("Annual interest rate in % (> 0): ");
    rate = decimal.Parse(Console.ReadLine());
}
while (rate <= 0);      // 0 % -> the loop below never ends!

```

```
decimal balance = principal;
int years = 0;
Console.WriteLine($"{"Year",4} {"Balance (TL)",14}");
while (balance < 2 * principal)
{
    balance += balance * rate / 100;   // one year of interest
    years++;
    Console.WriteLine($"{years,4} {balance,14:F2}");
}
Console.WriteLine($"The deposit doubles after {years} years.");
Console.WriteLine($"Rule of 72: about {72 / rate:F1} years");
```

Sample run 1

```
Deposit (TL): 10000
Annual interest rate in % (> 0): 0
Annual interest rate in % (> 0): 5
Year   Balance (TL)
   1       10500.00
   2       11025.00
   3       11576.25
   4       12155.06
   5       12762.82
   6       13400.96
   7       14071.00
   8       14774.55
   9       15513.28
  10       16288.95
  11       17103.39
  12       17958.56
  13       18856.49
  14       19799.32
  15       20789.28
The deposit doubles after 15 years.
Rule of 72: about 14.4 years
```

Sample run 2

```
Deposit (TL): 10000
Annual interest rate in % (> 0): 12
Year   Balance (TL)
   1       11200.00
   2       12544.00
   3       14049.28
   4       15735.19
   5       17623.42
   6       19738.23
   7       22106.81
The deposit doubles after 7 years.
Rule of 72: about 6.0 years
```

The loop finds the smallest whole number of years n with $(1 + r/100)^n \geq 2$, i.e.

$$n = \left\lceil \frac{\ln 2}{\ln(1 + r/100)} \right\rceil .$$

For $r = 5$: $\ln 2 / \ln 1.05 \approx 14.21$, so $n = 15$ ($1.05^{14} \approx 1.980 < 2 < 1.05^{15} \approx 2.079$). For $r = 12$: $\ln 2 / \ln 1.12 \approx 6.12$, so $n = 7$. The **rule of 72**, $n \approx 72/r$, is a quick mental estimate (14.4 and 6.0). The `do-while` validation rejects $r \leq 0$: with 0 % the balance would never change and the loop would never end.

>_ 11 Infinite loops and loop invariants

11.1 Infinite loops

A loop whose condition never becomes `false` runs forever. In the VS Code terminal you stop a running program with **Ctrl+C**. Typical causes:

(1) No update: prints 1 forever

```
int i = 1;
while (i <= 10)
{
    Console.WriteLine(i);
}
```

(2) Stray semicolon: hangs silently

```
int i = 1;
while (i <= 10);
{
    Console.WriteLine(i);
    i++;
}
```

(3) Condition never false

```
int k = 1;
while (k != 10)   // 1, 3, 5, ...
{
    k += 2;
}
```

(4) Nothing changes: rate 0 %

```
while (balance < 2 * principal)
{
    balance += balance * 0 / 100;
}
```

In (2) the semicolon is an *empty body*; the block below it is not part of the loop. The compiler warns: `warning CS0642: Possible mistaken empty statement`. In (3) `k` is always odd, so it is never equal to 10 (even after an overflow it stays odd). Debugging tip: print the loop variables inside the loop, or set a breakpoint and step through it.

11.2 Loop invariants (informally)

A **loop invariant** is a statement about the variables that is true before the loop, stays true after every pass, and — together with the exit condition — gives the result. It is how we convince ourselves (and the examiner) that a loop is correct for *every* input, not only for the one we tested.

Loop	Invariant (before each test)	At the end
sum $1..n$: $sum \leftarrow sum + i, i \leftarrow i + 1$ while $i \leq n$	$sum = 1 + 2 + \cdots + (i-1)$	$i = n + 1$, so $sum = 1 + \cdots + n$
Euclid: $(x, y) \leftarrow (y, x \bmod y)$ while $y \neq 0$	$\gcd(x, y) = \gcd(a, b)$	$y = 0$ and $\gcd(x, 0) = x$
halving (Example 10)	$low \leq secret \leq high$	the range shrinks every pass, so the loop ends

>_ 12 Common mistakes

1. Semicolon after the while condition

`while (i <= 10);` has an empty body: an infinite loop if `i <= 10`. Warning CS0642. Remove the semicolon.

2. Missing semicolon after do-while

`do { … } while (x > 0)` without the final `;` gives `error CS1002: ; expected`.

3. No progress towards the exit

No statement in the body changes the variables of the condition, or the change never makes the condition false (`k += 2` with `k != 10`). Check: *what changes in each pass, and why will the condition eventually be false?*

4. Assignment instead of comparison

`while (x = 5)` does not compile in C#: `error CS0029: Cannot implicitly convert type 'int' to 'bool'`. Compare with ==.

5. Sentinel loop with the READ at the top

If the value is read at the *top* of the body, the sentinel -1 is added to the sum and counted. Use a priming read before the loop and the next READ as the last statement of the body.

6. Range check without !ok

After a failed `TryParse` the value is 0; if 0 is in range, invalid text is accepted. Test `!ok || out of range`.

7. Off-by-one and wrong loop kind

< instead of <= (or vice versa) gives one pass too many or too few; a `while` loop where the body must run at least once (or a `do-while` where zero passes must be possible, Example 1 with $n = 0$). Trace the boundary cases: 0, 1, the sentinel first, invalid text.

8. Using the variable you are destroying

Digit loops end with the number equal to 0. Work on a copy (`rest`) if you still need the original afterwards (palindrome test, printing).

>_ 13 Summary

- `while` tests **before** the body: 0 or more passes. `do-while` tests **after** the body: 1 or more passes, and ends with `while (…);`.
- Use `for` when the number of passes is known (counter-controlled); `while`/`do-while` when you only know when to stop (condition-controlled). Every `for` loop can be rewritten as a `while` loop (init before, condition in the header, update at the end of the body).
- Sentinel loops: priming read before the loop, the next read at the end of the body; check for "no data" after the loop.
- Validation: repeat while `!ok || out of range`. Menus: a `do-while` loop around a `switch`.
- `% 10` and `/ 10` peel off decimal digits; `% 2` and `/ 2` give binary digits. Euclid's GCD, the LCM, Collatz and binary search are `while` loops.
- `break` leaves a loop, `continue` jumps to the next test; `while (true)` needs a `break`.
- Every loop needs progress towards its exit. Stuck? Ctrl+C, then trace the variables. A loop invariant explains *why* a loop is correct.

>_ 14 Glossary (English – Turkish)

English	Turkish	Meaning
while loop	while döngüsü	pre-test loop: condition tested before each pass
do-while loop	do-while döngüsü	post-test loop: body runs at least once
pre-test / post-test loop	ön koşullu / son koşullu döngü	where the condition is tested
counter-controlled	sayaç kontrollü	number of passes known in advance
condition-controlled	koşul kontrollü	repeat until something happens
sentinel (value)	bitiş (nöbetçi) değeri	special input that means “stop”
priming read	ilk okuma	the read before a sentinel loop
input validation	girdi doğrulama	checking that input is acceptable
menu-driven program	menü tabanlı program	program that repeats a menu until Exit
infinite loop	sonsuz döngü	loop whose condition never becomes false
loop invariant	döngü değişmezi	statement true before and after every pass
flag	bayrak (değişkeni)	`bool` variable that records an event
digit	basamak / rakam	one decimal position of a number
greatest common divisor (GCD)	en büyük ortak bölen (EBOB)	largest number dividing both
least common multiple (LCM)	en küçük ortak kat (EKOK)	smallest positive common multiple
binary (base 2)	ikili (2 tabanlı) sayı	number written with 0 and 1
palindrome	palindrom (ters okunuşu aynı)	reads the same backwards
seed (random)	tohum (rastgele)	start value of a random number generator
binary search	ikili arama	halving the search range each step

>_ 15 Exercises

15.1 Check yourself: predict the output

Write the exact output and the number of passes of the body. Then run the code to check.

Q1

```
int n = 10;
while (n > 1)
{
    n = n / 2;
    Console.Write($"{n} ");
}
```

Q2

```
int x = 100;
do
{
    Console.Write($"{x} ");
    x += 10;
}
while (x < 50);
Console.WriteLine($"| {x}");
```

Q3

```
int c = 0;
int m = 1;
while (m < 100)
{
    m *= 3;
    c++;
}
Console.WriteLine($"{m} {c}");
```

15.2 In-class exercises (pairs)

E1. ⋆ **Even and odd.** Read integers until 0 (sentinel). Print how many of them were even and how many odd.

E2. ⋆ **Month days.** Read a month number 1..12 with `do-while` and `TryParse` (ask again if invalid). Print its number of days (February = 28) with a `switch`.

E3. ⋆⋆ **Count a digit.** Read $n \geq 0$ and a digit d. Print how many times d occurs in n (e.g. 1 in 10101 → 3). Be careful with $n = 0$.

E4. ⋆⋆ **From `for` to `while`.** Rewrite the Week 6 factorial loop as a `while` loop. Then find the smallest n with $n! > 1\,000\,000$.

15.3 Homework

H1. ⋆⋆ **Unit converter menu.** `do-while` + `switch`: 1) km → mile, 2) °C → °F, 3) kg → lb, 0) Exit. Validate every number with `TryParse`. (1 mile = 1.609344 km, 1 lb = 0.45359237 kg, $F = \frac{9}{5}C + 32$.)

H2. ⋆⋆ **Prime check with `while`.** Read $n \geq 2$ (validated) and test the divisors $d = 2, 3, \ldots$ while $d \cdot d \leq n$. Stop at the first divisor found.

H3. ⋆⋆⋆ **Better guessing game.** Example 9 with at most 7 attempts; after each game ask "Play again? (y/n)" with a `do-while` around the whole game.

H4. ⋆⋆ **Binary to decimal.** Read a number made of 0s and 1s (e.g. `1101`) as an `int` and convert it with `% 10` and `/ 10` (1101 → 13). Report an error if a digit is larger than 1.

15.4 Extra practice

X1. ⋆ **Trace table.** Trace the loop below with columns `a`, `b`, `t`, `b < 30`. What is printed and how many passes are there?

```
int a = 1;
int b = 1;
while (b < 30)
{
    int t = a + b;
    a = b;
    b = t;
}
Console.WriteLine($"{a} {b}");
```

X2. ⋆ **Convert.** Rewrite as a `while` loop and give the output: `for (int i = 10; i >= 1; i -= 3) Console.Write($"i ");`

X3. ⋆⋆ **Find the bugs.** The program should print the digit sum of n. Find three mistakes.

```
int n = int.Parse(Console.ReadLine());
int sum;
while (n > 0);
{
    sum = n % 10;
    n = n % 10;
}
Console.WriteLine(sum);
```

X4. ⋆⋆ **Digital root.** Repeatedly replace n by the sum of its digits until a single digit remains ($987654 \to 39 \to 12 \to 3$). Use a loop inside a loop.

X5. ⋆⋆ **Smallest power of 2.** Read $n \geq 1$ and find the smallest k with $2^k \geq n$ using a `while` loop.

X6. ⋆⋆⋆ **Perfect numbers.** A number is *perfect* if it equals the sum of its divisors smaller than itself ($28 = 1 + 2 + 4 + 7 + 14$). Test divisors only while $d \cdot d \leq n$ and add each divisor together with its partner n/d.

>_ 16 References

- Microsoft Learn — Iteration statements (`for`, `foreach`, `do`, `while`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements
- Microsoft Learn — Jump statements (`break`, `continue`, `return`, `goto`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/jump-statements
- Microsoft Learn — Selection statements (`if`, `switch`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements
- Microsoft Learn — `Int32.TryParse` method: https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse
- Microsoft Learn — `Random.Next` method and `Random.Shared`: https://learn.microsoft.com/en-us/dotnet/api/system.random.next https://learn.microsoft.com/en-us/dotnet/api/system.random.shared
- Microsoft Learn — `Convert.ToString` (to base 2): https://learn.microsoft.com/en-us/dotnet/api/system.convert.tostring